Residual Lean proof simplifications after prior adoption

DiophantineConsistency fifth review

26 October 2026 Frozen source `e1c8914bc706`

Result. Three small candidate families survive the final prior-work screen. They simplify two Sidon injectivity endpoints, reuse exact controller orders at exponent zero, and close a definitional iterator-depth equality directly. Together they save nine physical source lines. This is an incremental cleanup, not a new architecture or a large reduction. All proposed Lean code remains **uncompiled**.

ID	Candidate family	Complete file	Net saved
S	Sidon injectivity endpoints	168 → 164	4
O	Order-one specializations for controller powers	105 → 102	3
D	Definitional iterator-depth equality	163 → 161	2
	Total against pinned source	436 → 427	9

Assumed-applied accounting. The earlier Sidon moment-polarization patch already changes its file from 168 to 161 lines. Our endpoint patch commutes with it, giving 161 → 157, still four new lines saved. Thus the concrete composed baseline for these three files is 429 → 420, again nine. The prior seven-line reduction is not claimed again. These are physical source-line counts, not proof-term sizes, elaboration times, or an assertion that every prior design has been materialized into one compatible checkout.

What stays unchanged. All three candidates preserve the existing public theorem statements, definitions, constants, and imports. There is no new helper API and no change to the finite physical data, machine, controller modulus, expressions, or witness counts. The Sidon changes are one family, despite touching two endpoints; the two controller endpoints are likewise one family.

Source and novelty boundary. The source snapshot contains 690 active Lean files, including checks. The review grants all earlier collection proposals and this report series' four delivered rounds as already applied, whether or not their production implementation exists. The initial refreshed collections contain a–w and five incoming ZIP packets; the closing screen expands this to a–zj with no remaining incoming ZIPs. Broad previously promised adapters are not counted as fresh work merely because their code is absent.

Validation boundary. Source hashes, exact patches, preserved declaration interfaces, finite models, and document rebuilds can be checked here. Lean and Lake are absent and were not installed. No new kernel elaboration, theorem axiom output, full-project integration build, or performance measurement is claimed. No upstream file was changed.

Recommendation. Test O if removing repeated endpoint arguments is worth the stronger proof dependency; its specializations use an interface already present in the same file. S and D are optional local cleanup. Keep the existing regression checks and compare endpoint axioms before adoption.

Closing screen The one closing check is frozen at `91dce223cda86c0755b8a9b2504f84be04443fce`, 18:37:26 UTC. It adds 13 research directories x–zj: five relocate the already-inspected incoming archives, zi is the previous delivered round, and seven contain further proposals. Those seven manuscripts and 37 supplementary notes were screened against the shortlist. F is withdrawn as an exact prior proposal. S, O and D remain distinct at this cutoff. No changes after that closing pin are covered.

1 Recent commits and the refreshed baseline

The frozen commit is `e1c8914bc7064d9a8e7c06171a18069a949badd0`, at 18:30:35 UTC. It is 47 commits ahead of the previous source pin `e652d21c413de59e4821f6e8e9b6a3a077bf5bfe`, and 25 ahead of the previous closing screen `09d65a93c9df8684c1a226341d6a5120623d1cdb`. Complete Git trees, not a possibly capped compare-file array, determine the active changes.

The active Lean tree grows from 637 to 690 files: 53 added, 32 modified, none removed. Against the previous closing screen the corresponding figures are 664 to 690, with 26 added and 26 modified. Physical source lines grow from 80,328 to 86,351. The net 6,023-line increase combines new constructions and refactors; it is not a simplification metric.

1.1 Observed adoption is not a new recommendation

The recent history already incorporates signed root recovery, pairing projection facts, coefficient envelopes, decoder invariants, scanner induction, shared isolated coefficient calculations, and internal arithmetic/model extraction. Representative commits are `025c99b12c07` (signed roots), `15295203798b` (pairing projections), `4de06108b31c` (coefficient envelopes), `fad90f705b0b` (quadratic fibers), and `dbc6446fa790` (scanner inductions). Their savings belong to earlier work.

Whole-file reductions such as NativeCoefficientBounds 679 → 127 and ObjectCodeBound 476 → 435 are reported only as source observations. They are not attributed to this review and are not interchangeable with net proposal savings after adding helpers.

1.2 Additional exclusions at this pin

All 23 a–w directory trees are rebound to the current pin. The four incoming archives inspected at the preceding closing screen are unchanged. A fifth, `ProveIt_Additional_Lean_Simplifications_2026-10-26.zip`, adds six excluded mechanisms:

1. periodic-orbit helical transport;
2. combined-divisibility cancellation in ratio reparameterization;
3. the geometric-series identity and its upper-bound consequence;
4. deriving coordinate inequalities from positive square margins;
5. exact signed-prefix division/remainder and borrow interfaces, including its proposed concatenation theme;
6. rank comparison through $C + qF$, with the actual carrier adapters.

The previous round's shared halted-color argument is also granted. Temporal shared-edge grids, exact canonical band quotient reuse, word support normalization, copied-field descriptors, carry uniqueness, generic coefficient fibers, and automatic scanner/decoder frameworks remain excluded.

Evidence. `provenance/COMMIT_AND_SOURCE_SUMMARY.md`, `provenance/commit_timeline.json`, and `baseline_audit/` distinguish the source delta from the assumed-applied proposal ledger. The latter is a semantic baseline, not a claim that all conflicting old patches can be applied simultaneously.

2 S Normalize the two Sidon injectivity endpoints

Source anchors. `SidonPositions.lean` lines 94–100 and `SidonPositions.lean` lines 151–162. This proposal consumes the unchanged statement of `weight_sum_eq_iff`. The earlier moment-polarization simplification of that theorem is already assumed applied.

2.1 Use a diagonal pair rather than a distinguished zero

The original proof adds `weight m 0` to both sides, then treats direct and swapped unordered-pair cases differently. Instead, add each weight to itself:

```
theorem weight_injective (m : ℕ) : Function.Injective (weight m) := by
  intro i j h
  rcases (weight_sum_eq_iff i i j j).mp
    (congrArg (fun n => n + n) h) with h | h <;>
    exact h.1
```

Both disjuncts now have the identical first component $i = j$. No property of the zero position is used. The literal supplied patch is two lines shorter than the original body.

2.2 Let ordered endpoints discharge the swapped case

For canonical pairs $p = (a, b)$ and $q = (c, d)$, the retained subtype hypotheses give $a \le b$ and $c \le d$. In the swapped case the Sidon theorem gives $a = d$ and $b = c$. Hence $a \le b = c \le d = a$, so both pairs are equal. The explicit chain of two rewrites, antisymmetry, and transitivity can be replaced by linear arithmetic after extensionality:

```
theorem pairWeight_injective (m : ℕ) :
    Function.Injective (pairWeight (m := m)) := by
  intro p q h
  rcases (weight_sum_eq_iff p.val.1 p.val.2 q.val.1 q.val.2).mp h with
    ⟨h1, h2⟩ | ⟨h1, h2⟩
  · exact Subtype.ext (Prod.ext h1 h2)
  · have hp := p.property
    have hq := q.property
    apply Subtype.ext
    apply Prod.ext <;> apply Fin.ext <;> omega
```

Accounting and compatibility. These are one endpoint-normalization family, with four new lines removed in total. The pinned complete file is 168 → 164. The actual earlier moment-polarization patch composes in either order, without fuzz, to the same bytes: 161 → 157 relative to the assumed-applied baseline. Neither its imports nor its seven-line reduction is counted again. The pair encoding, repeated-index cases, and quadratic coefficient theorem remain unchanged.

Tradeoff and gate. The mathematics is elementary, but the last line relies on Lean's `omega` preprocessing of finite indices and equalities. That exact elaboration is not tested here. If it needs explicit `congrArg Fin.val` facts, include those lines in a revised count rather than retaining the advertised four-line saving. Finite counterexample controls show why both ordering hypotheses matter; they do not certify the tactic script.

3 O Reuse exact order at exponent zero

Source anchors. `CurrentPowerCoordinates2011.lean` lines 46–67; the imported exact-order family in `CurrentAuxiliaryOrder2011.lean` lines 52–60. The two generator identities at the first prime-power level are already consequences of stronger exact-order theorems. At exponent zero the stated order is $p^0 = 1$, so the existing `orderOf_eq_one_iff` API returns the identity element directly.

```
theorem generator_cast_auxiliary :
    (B : ZMod auxiliary)^(auxiliary-1) = 1 := by
  apply orderOf_eq_one_iff.mp
  simpa only [Nat.zero_add, pow_one, pow_zero] using generator_order_succ 0

theorem generator_cast_C0 :
    (B : ZMod (d*auxiliary))^(auxiliary-1) = 1 := by
  apply orderOf_eq_one_iff.mp
  simpa only [Nat.zero_add, pow_one, pow_zero] using full_order 0
```

The first avoids an independent modular cast calculation; the second avoids recomputing the CRT product order at the base case. The complete file goes from 105 to 102 lines, with no new helper or import.

Dependency direction matters. `generator_order_succ` is imported from the auxiliary-order file. `full_order` occurs before `generator_cast_C0`; its proof uses `generator_cast_fixed` and `generator_order_succ`, not `generator_cast_auxiliary`. Thus the proposed reuse is acyclic. Both public base-case theorems are retained even though the second no longer needs the first. This does not replace or weaken the nontrivial prime-square lift needed for exact higher order. The first endpoint now depends on that stronger exact-order argument rather than only the elementary residue calculation. This is a maintenance option, not dependency minimization; no axiom-footprint or runtime improvement is asserted.

4 D Close a definitional depth equality directly

Source anchors. `IterationSize.lean` lines 68–74. The existing proof first establishes equality of the two expressions by `rfl`, then applies `congrArg Expr.depth`. But depth is already the same expression projection on both sides. The proposed complete theorem is:

```
theorem iterate_depth {f : ℕ → ℕ} (p : R f) (initial : ℕ) :
    (DirectGraphEnumerator.iterate p initial).depth =
      (rows p initial).depth := rfl
```

This removes two physical lines in the supplied file, 163 → 161. It changes no expression, proof-bearing representation, iterator, or witness allocation. It is a small definitional-equality cleanup, not a new metadata abstraction.

Novelty boundary. Earlier work moves witness-count projections before their users and factors depth-budget arguments; those outcomes are granted. This proposal only deletes the remaining congruence proof after a definitional equality. If a future refactor makes the projections opaque or not definitionally equal, keep the previous proof or update this candidate; no performance advantage is asserted.

5 Reproducibility and integration gates

The archive is a focused research snapshot, not a complete Lake checkout. The original source manifest binds 697 files: all 690 active Lean modules and seven configuration/context files. A separate manifest binds 27 direct external repository dependencies; another binds 32 predecessor versions. These are not a transitive dependency closure. Mathlib evidence for the excluded comparison F is tied to the project's exact revision `81a5d257c8e410db227a6665ed08f64fea08e997`.

1. Run `python3 verify_bundle.py --strict` to detect added, missing, or changed delivered files.
2. Run `python3 provenance/verify_snapshot.py` to recheck raw Git tree hashes, blob hashes, SHA256 values, active coverage, and exact source deltas.
3. Run `python3 -m unittest discover -s validation -p 'test_*.py'` for source-preservation and bounded-model checks. Run the controller review's `verify.py` for its exact patch and modular checks.
4. Regenerate local candidate files with `local_candidates/build_candidates.py`; these are proof-text transformations only. The included receipts compare patch outputs and regenerated bytes.
5. In a separate complete checkout at the source pin, use the supplied replay plan to compile full edited modules and the corresponding existing checks, then compare endpoint axioms. This step is **not run** here.
6. Rebuild the PDF with `bash report/build.sh`; the release audit compares every page's extracted text and rendered pixels, not byte identity of newly generated PDF metadata.

5.1 What the non-Lean checks establish

Exact source transformations preserve the surrounding modules and theorem signatures. Finite models test finite-order rigidity, diagonal and sorted-pair identities, and guard necessity. Modular arithmetic confirms representative controller constants without re-evaluating them inside Lean. Such tests can expose a wrong arithmetic idea or a missing guard; they cannot establish kernel acceptance for arbitrary parameters.

The earlier finite-rigidity candidate is excluded: newly placed packet zh supplies the identical proof and packet zj an alternative. Its code and finite-order models remain labeled comparison evidence only, with zero new savings credited.

The independent source review also checks the concrete Sidon composition with the assumed earlier patch. Every candidate must still pass elaboration, existing regression checks, and an axiom comparison. The whole repository and all reverse dependencies have not been built.

5.2 Further research questions

1. Does O preserve the original endpoint axioms after routing through exact-order results? If not, is the lighter elementary proof preferable despite three extra source lines?
2. Does the exact `omega` script in S elaborate under the pinned imports without extra conversion facts? Count any required facts before adoption.
3. Do these small wrappers measurably improve elaboration or maintenance after the much larger earlier abstractions are integrated? Source-line savings alone do not answer this.

Negative findings. Repeated residue chunks remain covered by checked finite dispatch. Terminal word support and reflected origin zeros repeat earlier support/fiber APIs. Generic carry reset factoring, copied-field parity sharing, and broad ZMod coercion cleanup also repeat the assumed baseline. No new large arithmetic abstraction is claimed merely to increase the number of recommendations.